

Biocompiler 1.0

DNA Transcriber — Especificação da Atividade

Objetivo: receber uma sequência de DNA e simular, de forma didática, sua análise e transcrição até a geração de um pré-mRNA.

1. Papel do BioCompiler 1.0

Entrada: sequência de DNA.

Processamento: validação da sequência, identificação da região codificante e transcrição do DNA em pré-mRNA.

Saída: pré-mRNA, quando a entrada for válida; caso contrário, um diagnóstico do problema.

Fase I: DNA transcrito em pré-mRNA — [BioCompiler 1.0](#) (objeto deste trabalho)

Fase II: pré-mRNA amadurecido em mRNA maduro — [BioCompiler 2.0](#)

Fase III: mRNA traduzido em proteína — [Sr. Ribossomo](#) (trabalho futuro)

2. Formato de entrada

A **entrada** será um **arquivo texto** (.txt) contendo uma ou mais sequências de DNA. Cada linha do arquivo corresponde a uma sequência independente. O programa deverá processar as linhas na ordem em que aparecem no arquivo.

Cada sequência deverá utilizar apenas A, T, C e G, sem espaços ou separadores internos.

Exemplo

Nome: *arquivo entrada.txt*

```
CCCATGGCTAAACCGTAA
GGGATGCCCGGGTAG
TTTATGAAACCCGGGTGA
```

3. Convenção didática para análise da sequência

Para tornar o problema implementável e corrigível automaticamente, o BioCompiler 1.0 usará uma gramática simplificada. Ela representa conceitos reais da expressão gênica, mas não pretende reproduzir toda a complexidade biológica.

REGIÃO CODIFICANTE VÁLIDA: **ATG** ... **TAA / TAG / TGA**

Elemento	Representação	Função didática
START	ATG	Marca o início da região codificante.
Códons internos	trincas de A/T/C/G	Mantêm a leitura em grupos de três bases.
STOP	TAA, TAG ou TGA	Marca o término da região codificante.

4. Regra de identificação da região codificante

Para a atividade, o programa deverá localizar o primeiro ATG válido e, a partir dele, ler a sequência em trincas. O término válido será o primeiro códon TAA, TAG ou TGA encontrado no mesmo quadro de leitura.

Bases antes do ATG e depois do STOP podem existir e não fazem parte da região codificante usada para a validação do quadro de leitura.

5. Operação de transcrição

Quando a sequência for considerada válida, o programa deverá transcrever o DNA para pré-mRNA substituindo cada timina (T) por uracila (U), preservando a ordem das demais bases.

DNA: ATGGCTAAACCGTAA

pré-mRNA: AUGGCUAAACCGUAA

6. Convenção didática para frameshift

Para esta atividade, será classificado como *frameshift* o caso em que, após o ATG, a região analisada não puder ser organizada integralmente em trincas até o término esperado. Essa é uma convenção computacional do exercício.

Observação: biologicamente, a identificação inequívoca de uma mutação *frameshift* exige comparação com uma sequência de referência. Aqui, o termo é usado segundo a regra didática acima.

7. Convenção didática para nonsense / STOP prematuro

Para a atividade, será considerado **STOP prematuro** quando surgir um códon TAA, TAG ou TGA em fase antes do término esperado da região codificante, havendo ainda sequência codificante após esse ponto. Essa é uma convenção didática criada para permitir classificação automática

Observação: biologicamente, determinar que um STOP é prematuro também requer contexto ou sequência de referência. No BioCompiler 1.0, prevalece a convenção estabelecida no exercício.

8. Casos que o programa deverá reconhecer

Caso	Situação	Resposta esperada
1	Sequência válida	CORRETO
2	Presença de caractere diferente de A, T, C ou G	BUG - base inválida
3	ATG de início ausente	BUG - START ausente
4	STOP válido ausente	BUG - STOP ausente
5	Quadro de leitura incompatível com trincas segundo a convenção	BUG - frameshift
6	STOP em fase antes do término esperado	BUG - nonsense / STOP prematuro

9. Saída padrão para a tela

A saída exibida na tela será detalhada e destinada ao usuário. Para cada linha, o programa mostrará o número da entrada, o status, o diagnóstico e, quando aplicável, o pré-mRNA.

```
=====
BIOCOMPILER 1.0 - DNA TRANSCRIBER
=====
ENTRADA: 1
STATUS: CORRETO
Bases: OK
START: ATG - OK
Quadro de leitura: OK
STOP: OK
Transcrição: OK
pré-mRNA: AUGGCUAAACCGUAA
-----
```

10. Processamento em lote

O arquivo poderá conter uma ou várias linhas. O programa deverá processar cada linha de forma independente, preservar a ordem das entradas e gerar um resultado correspondente para cada uma delas.

11. Saída padrão para arquivo exportado

Além da saída na tela, o programa deverá gerar um arquivo de resultados. Para facilitar a correção automática, recomenda-se um registro por linha, com campos separados por ponto e vírgula.

```
linha;status;resultado;pre_mRNA
1;OK;CORRETO;AUGGCUAAACCGUAA
2;ERRO;BUG - base inválida;NÃO GERADO
3;ERRO;BUG - START ausente;NÃO GERADO
```

Padrão proposto: arquivo resultados.txt (ou .csv, se desejado), codificação UTF-8. Os textos da coluna 'resultado' devem ser reproduzidos exatamente como definidos na especificação.

12. Requisitos de implementação

- Linguagens aceitas: Java ou Python.
- O programa deve ler as entradas a partir de arquivo texto.
- Uma entrada por linha.
- Não assumir previamente qual caso está sendo analisado.
- Não utilizar separadores internos na sequência de entrada.
- O BioCompiler 1.0 termina no pré-mRNA; ele não realiza maturação do RNA nem tradução de proteínas.

13. Resumo do algoritmo

```
LER linha
↓
VALIDAR caracteres A/T/C/G
↓
LOCALIZAR START (ATG)
↓
ANALISAR quadro de leitura em trincas
↓
LOCALIZAR e VALIDAR STOP
↓
CLASSIFICAR a sequência
↓
SE CORRETA: TRANSCREVER T → U
↓
GERAR pré-mRNA + RELATÓRIO
```

Princípio da atividade: o aluno deve implementar um analisador molecular, não memorizar casos de teste.

14. Exemplos de entrada e saída por caso

Suponha um arquivo de entrada com SEIS linhas, cada linha correspondendo a uma sequência de DNA. A primeira é válida e as demais representam, didaticamente, cada uma das situações que o BioCompiler 1.0 deverá reconhecer. A seguir são mostradas a saída na tela e a forma correspondente no arquivo exportado.

14.1 CORRETO

A sequência contém **START**, quadro de leitura e **STOP** válidos.

Entrada: ATGGCTAAACCGTAA

Saída na tela:

```
ENTRADA: 1
STATUS: CORRETO
Bases: OK
START: ATG - OK
Quadro de leitura: OK
STOP: TAA - OK
Transcrição: OK
pré-mRNA: AUGGCUAAACCGUAA
```

Registro no arquivo exportado:

```
1;OK;CORRETO;AUGGCUAAACCGUAA
```

14.2 BUG - base inválida

A sequência contém um caractere que não pertence ao alfabeto A/T/C/G.

Entrada: ATGGCTXAACCGTAA

Saída na tela:

```
ENTRADA: 2
STATUS: ERRO
TIPO: BUG - base inválida
pré-mRNA: NÃO GERADO
```

Registro no arquivo exportado:

```
2;ERRO;BUG - base inválida;NÃO GERADO
```

14.3 BUG - START ausente

A sequência não apresenta o códon ATG necessário para iniciar a região codificante.

Entrada: CCCGCTAAACCGTAA

Saída na tela:

```
ENTRADA: 3
STATUS: ERRO
TIPO: BUG - START ausente
pré-mRNA: NÃO GERADO
```

Registro no arquivo exportado:

```
3;ERRO;BUG - START ausente;NÃO GERADO
```

14.4 BUG - STOP ausente

Após o START, não é encontrado TAA, TAG ou TGA válido no quadro de leitura.

Entrada: ATGGCTAAACCGGGC

Saída na tela:

```
ENTRADA: 4
STATUS: ERRO
TIPO: BUG - STOP ausente
pré-mRNA: NÃO GERADO
```

Registro no arquivo exportado:

```
4;ERRO;BUG - STOP ausente;NÃO GERADO
```

14.5 BUG - frameshift

Segundo a convenção didática, a região após o START perde a organização esperada em trincas até o término.

Entrada: ATGGCTAAAACCGTAA

Saída na tela:

```
ENTRADA: 5
STATUS: ERRO
TIPO: BUG - frameshift
pré-mRNA: NÃO GERADO
```

Registro no arquivo exportado:

```
5;ERRO;BUG - frameshift;NÃO GERADO
```

14.6 BUG - nonsense / STOP prematuro

Um STOP em fase aparece antes do término esperado da região codificante, restando sequência após ele.

Entrada: ATGGCTTAACCGGGCTAA

Saída na tela:

```
ENTRADA: 6
STATUS: ERRO
TIPO: BUG - nonsense / STOP prematuro
pré-mRNA: NÃO GERADO
```

Registro no arquivo exportado:

```
6;ERRO;BUG - nonsense / STOP prematuro;NÃO GERADO
```

PRÓXIMA PISTA:

EXERCÍCIO PROPOSTO: BIOCOMPILER

Sherlock DNA e Watson desafiam você a transformar conhecimento biológico em solução computacional!



SUA MISSÃO:

Desenvolver um programa que simule o **fluxo** da informação genética:



DNA



RNA



PROTEÍNA

- ✓ Valide, **transcreva** e traduza sequências.
- ✓ Detecte **mutações** e seus efeitos.
- ✓ Projete um **RNAi** e avalie a especificidade.
- ✓ Pense como um(a) cientista... programe como um(a) desenvolvedor(a)!

PREPARE SEU CÓDIGO.

</> A VIDA DEPENDE DA INFORMAÇÃO! </>

BIOCOMPILER

Transforme o código da vida em lógica de programação e descubra padrões que podem salvar vidas!



MISSÃO ACEITA?

```
DNA = "ATG GAA CCG TAA"  
mRNA = transcrever(DNA)  
proteina = traduzir(mRNA)  
print(proteina)
```

```
> Met - Glu - Pro - STOP
```

CIÊNCIA
+
CÓDIGO
=
IMPACTO

BIOLOGIA MOLECULAR

GENÉTICA

BIOINFORMÁTICA

CIÊNCIA DA COMPUTAÇÃO



BIOCOMPILER 1.0

BIOCOMPILER 1.0 – DNA TRANSCRIBER

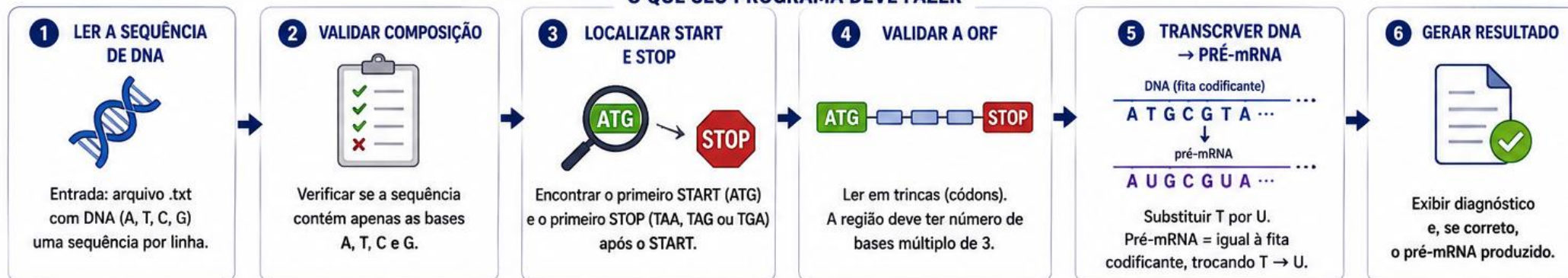
SLIDE 1/2: VISÃO GERAL – DNA → PRÉ-mRNA (TRANSCRIÇÃO)



Sua missão: implementar o BioCompiler 1.0.

Seu programa deve **transcrever** (copiar) uma sequência de **DNA** em um **pré-mRNA**.

O QUE SEU PROGRAMA DEVE FAZER



REGRAS DO BIOCOMPILER 1.0



Usar apenas as bases A, T, C e G.



START = códon ATG.



STOP = códons TAA, TAG ou TGA.



A leitura é feita de 3 em 3 bases (códon), a partir do START.



A região entre START e STOP (inclusive) deve ter múltiplo de 3 bases.

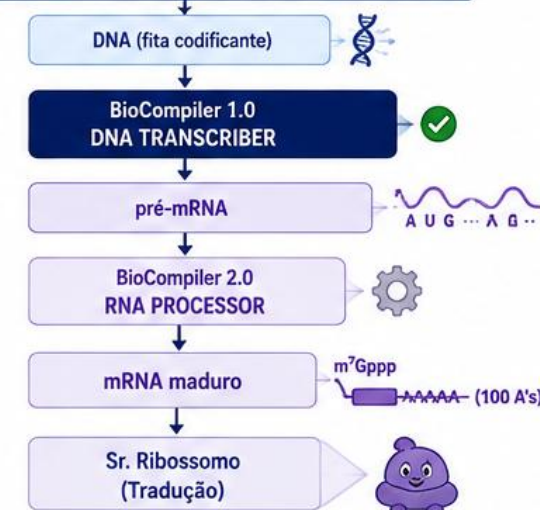


A transcrição gera o pré-mRNA correspondente à fita codificante: A → A, T → U, C → C, G → G.

O QUE SEU PROGRAMA DEVE IDENTIFICAR

CASO	SITUAÇÃO	RESPOSTA ESPERADA
✓ 1	Sequência válida	CORRETO
⚠ 2	Contém base diferente de A, T, C ou G	BUG - base inválida
⚠ 3	START (ATG) ausente	BUG - START ausente
⚠ 4	STOP (TAA, TAG ou TGA) ausente após o START	BUG - STOP ausente
⚠ 5	Número de bases entre START e STOP não é múltiplo de 3 (frameshift)	BUG - frameshift
❓ 6	STOP prematuro (TAA, TAG ou TGA) antes do primeiro STOP válido na ORF	BUG - nonsense / STOP prematuro

ONDE ESTAMOS NO FLUXO DO PROJETO



IMPORTANTE: A entrada contém apenas as letras A, T, C e G (maiúsculas). Não há espaços, quebras de linha internas ou outros símbolos.



PENSE COMO UM COMPILADOR. ATUE COMO UM CIENTISTA!



Objetivo desta etapa: transformar DNA em pré-mRNA válido para que o BioCompiler 2.0 possa processá-lo.



DNA TRANSCRIBER

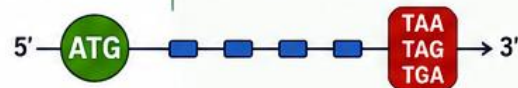
BIOCOMPILER 1.0 – DNA TRANSCRIBER

SLIDE 2/2: REGRAS E EXEMPLOS DOS 6 CASOS



CONVENÇÃO DIDÁTICA

Este trabalho usa uma convenção simplificada para tornar o problema implementável e corrigível automaticamente.



ATG START = início da ORF (códon ATG).

TAA TAG TGA STOP = final da ORF (códon TAA, TAG ou TGA).

A leitura é feita em trincas (códon) a partir do primeiro ATG encontrado.

REGRAS DO BIOCOMPILER 1.0

ATG START = códon ATG.

TAA TAG TGA STOP = códon TAA, TAG ou TGA.

[] [] [] A leitura é feita em trincas (códon), a partir do primeiro START.

3n A região entre START e STOP (inclusive) deve ter número de bases múltiplo de 3.

U Na transcrição, substituir T por U.

! Se houver erro em qualquer regra, o pré-mRNA NÃO é gerado.

FORMATO DE ENTRADA

Arquivo texto (.txt)
Uma sequência por linha.
O programa processa as linhas na ordem em que aparecem no arquivo.

O QUE SEU PROGRAMA DEVE FAZER

1 LER A SEQUÊNCIA DE DNA



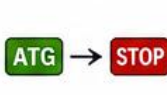
Uma sequência por linha.

2 VALIDAR COMPOSIÇÃO



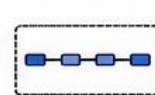
A sequência deve conter apenas A, T, C e G.

3 LOCALIZAR START E STOP



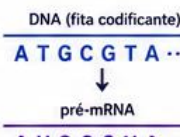
Encontrar o primeiro START (ATG) e o primeiro STOP (TAA, TAG ou TGA) após o START.

4 VALIDAR A ORF



A região entre START e STOP (inclusive) deve ter número de bases múltiplo de 3.

5 TRANSCRVER DNA → PRÉ-mRNA



Substituir T por U.

6 GERAR RESULTADO

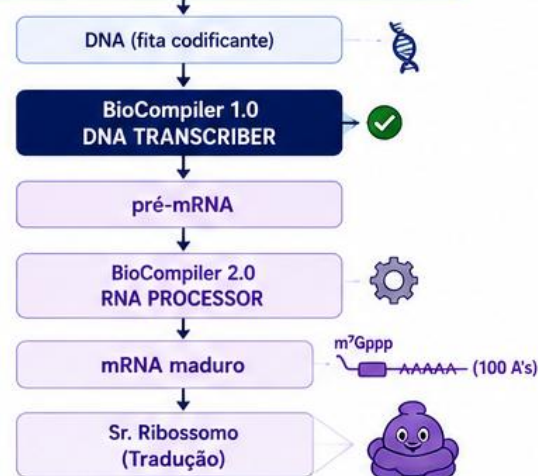


Exibir diagnóstico e, se correto, o pré-mRNA produzido.

OS 6 CASOS QUE SEU PROGRAMA DEVE RECONHECER

CASO	SITUAÇÃO	CRITÉRIO PRINCIPAL	RESPOSTA ESPERADA
✓ 1	Estrutura válida	Todas as regras atendidas.	CORRETO
! 2	Base inválida	Sequência contém caractere diferente de A, T, C ou G.	BUG - base inválida
! 3	START ausente	Não existe ATG na sequência.	BUG - START ausente
! 4	STOP ausente	Não existe STOP (TAA, TAG ou TGA) após o primeiro START.	BUG - STOP ausente
! 5	Frameshift	A região entre START e STOP tem número de bases que NÃO é múltiplo de 3.	BUG - frameshift
! 6	Nonsense / STOP prematuro	Existe STOP (TAA, TAG ou TGA) antes do primeiro STOP válido na ORF.	BUG - nonsense / STOP prematuro

ONDE ESTAMOS NO FLUXO DO PROJETO



SAÍDA DO PROGRAMA

Na tela
Para cada linha:
• Número da entrada
• Status / diagnóstico
• E, se correto, o pré-mRNA gerado

Em arquivo (resultados.txt ou .csv)
Formato recomendado (ponto e vírgula):
linha;status;resultado;pre_mRNA
Ex.: 1; OK; CORRETO;AUGC...UA (100 A's)
2; ERRO;BUG - base inválida;NÃO GERADO

OBJETIVO DESTA ETAPA

Transformar DNA em pré-mRNA válido e confiável para que o BioCompiler 2.0 possa realizar o processamento (splicing, CAP 5' e cauda poli-A).